

FIAP POSTECH — SOFTWARE ARCHITECTURE (SOAT)

Tech Challenge — Fase 3

Documento de Entrega

Sistema de Gestão de Oficina Mecânica

Evolução para nuvem, API Gateway, autenticação serverless e observabilidade — Azure · API Management ·
Azure Functions · AKS · Datadog

1. Participantes

Nome Completo	Username Discord	RM
Günter Zaffalon	zaffinha	rm372056
Gustavo Gouveia Olier Silva	gustavolierr	rm374227
Rafael Hernane Ramalho	ramalho.rafa	rm367128

2. Links do Projeto

Recurso	Link
Repositório — API principal	https://github.com/Gustavollier/postech-app
Repositório — Function de autenticação	https://github.com/Gustavollier/postech-auth-function
Repositório — Infraestrutura de banco de dados	https://github.com/Gustavollier/postech-infra-db
Repositório — Infraestrutura de nuvem (AKS, APIM, observabilidade)	https://github.com/Gustavollier/postech-infra-k8s
Collection das APIs (OpenAPI/Swagger)	https://github.com/Gustavollier/postech-app/blob/main/Documents/openapi.json
Vídeo de Demonstração	https://www.youtube.com/watch?v=Awv2vRF1krl

Os quatro repositórios estão públicos no GitHub. A entrega é composta pelo conjunto dos quatro — cada um com seu próprio pipeline de CI/CD — e não apenas pelo repositório da API principal.

3. Contexto e Objetivos da Fase 3

Depois da Fase 2 — que consolidou a aplicação com Clean Architecture, containerização, Kubernetes com HPA e um pipeline único de CI/CD — o cenário de negócio evoluiu: a oficina expandiu para múltiplas unidades físicas e ampliou sua base de clientes. Isso exigiu evoluir a arquitetura para garantir segurança de acesso, escalabilidade de operação em nuvem gerenciada e observabilidade de ponta a ponta. Os objetivos desta fase foram:

- Publicar a aplicação em um provedor de nuvem público, atrás de um API Gateway que centralize autenticação, rate limiting e correlação de requisições;
- Autenticar clientes por CPF através de uma função serverless dedicada, sem exigir senha, distinta do login tradicional de funcionário;
- Segregar a solução em repositórios independentes — aplicação, função de autenticação, infraestrutura de banco e infraestrutura de nuvem — cada um com seu próprio ciclo de CI/CD;
- Instrumentar observabilidade de ponta a ponta (logs, métricas, traces, dashboards e alertas);

- Registrar formalmente, através de RFC's e ADRs, as decisões técnicas e arquiteturais tomadas para sustentar essa operação em escala corporativa, incluindo a justificativa e os ajustes necessários no modelo de dados relacional.

O que foi entregue nesta fase

Frente	Entrega
Gateway de API	Azure API Management na frente de tudo: valida o JWT, aplica rate limiting por IP, propaga X-Correlation-ID e roteia para a API (AKS) ou para a Function de autenticação (Container Apps)
Autenticação por CPF	Function serverless dedicada (postech-auth-function), sem senha, que autentica o cliente e emite o mesmo tipo de JWT que a API aceita
Banco gerenciado	Azure SQL Database, com schema e segredos administrados por Terraform próprio (postech-infra-db)
Autorização granular	Dois emissores de token distinguidos por role; rotas de operação exigem cargo de equipe; rotas de consulta do cliente passam por checagem de posse, para que um cliente nunca leia dado de outro
Observabilidade	Datadog: Cluster Agent com admission controller instrumentando a API automaticamente, dashboards, monitors e synthetics
CI/CD	Pipeline próprio por repositório, com login na Azure via OIDC (sem segredo de nuvem versionado) e, na API principal, um smoke test que fala com o ambiente publicado de ponta a ponta

4. Desenho da Arquitetura

4.1 Arquitetura da aplicação (Clean Architecture)

A aplicação principal (postech-app) segue os princípios da Clean Architecture: a regra de dependência aponta sempre para o centro — o domínio não conhece nenhuma camada externa, e frameworks, banco de dados e web são detalhes nas bordas. Os repositórios são definidos como ports (interfaces) no domínio e implementados como adapters na camada de infraestrutura (Dapper + Azure SQL Database). A aplicação roda em .NET 10.

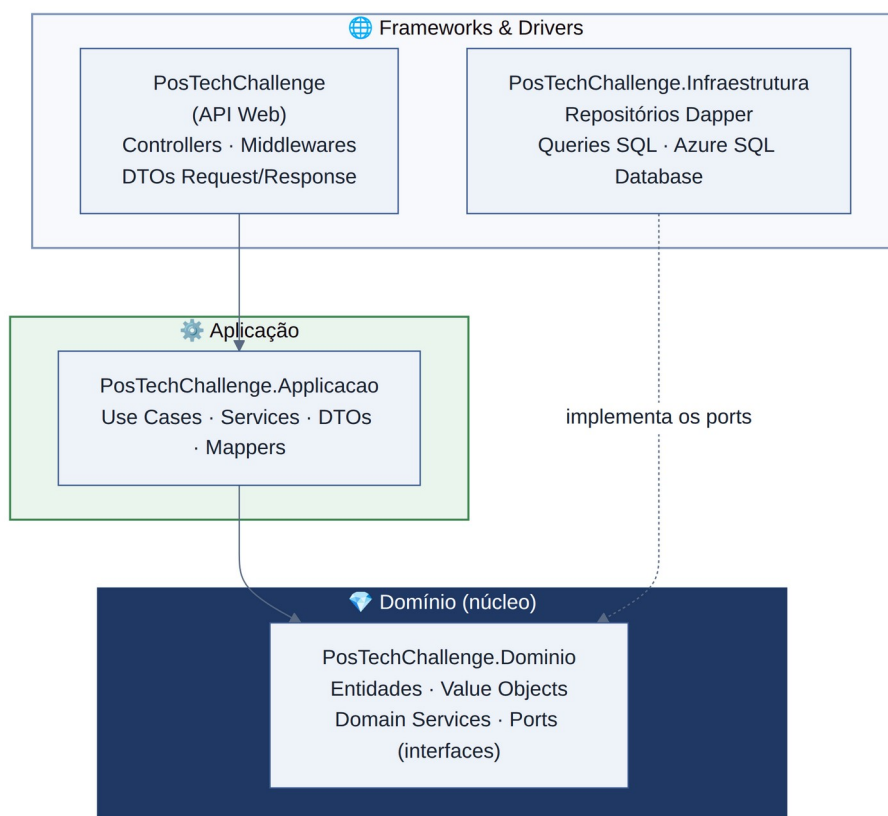


Figura 1 — Camadas da aplicação e regra de dependência

Camada	Projeto	Responsabilidade
API / Apresentação	PosTechChallenge	Controllers REST, Middlewares (correlação, rate limiting, timing), autorização por papel e por posse
Aplicação	PosTechChallenge.Applicacao	Use Cases, Application Services, DTOs, Mappers
Domínio	PosTechChallenge.Dominio	Entidades, Value Objects (CPF, CNPJ, Senha, Placa), Domain Services (máquina de estados da OS), Ports
Infraestrutura	PosTechChallenge.Infraestrutura	Repositórios Dapper, Queries SQL, Unit of Work
Testes	PosTechChallenge.Testes / .Testes.Integracao	Testes unitários (xUnit + Moq) e de integração contra SQL Server real

4.2 Repositórios da solução

O desafio da Fase 3 pede quatro repositórios com responsabilidades e pipelines separados. O contrato entre eles não é um arquivo formal, mas dois mecanismos concretos: o Azure Key Vault criado por postech-infra-db é a fonte única da connection string e do segredo JWT, lida por Terraform tanto de postech-infra-k8s quanto da configuração da Function — nenhum dos dois gera ou versiona segredo por conta própria; e o Azure Container Registry, criado em postech-infra-k8s, é o ponto de encontro das imagens publicadas por postech-app e por postech-auth-function.

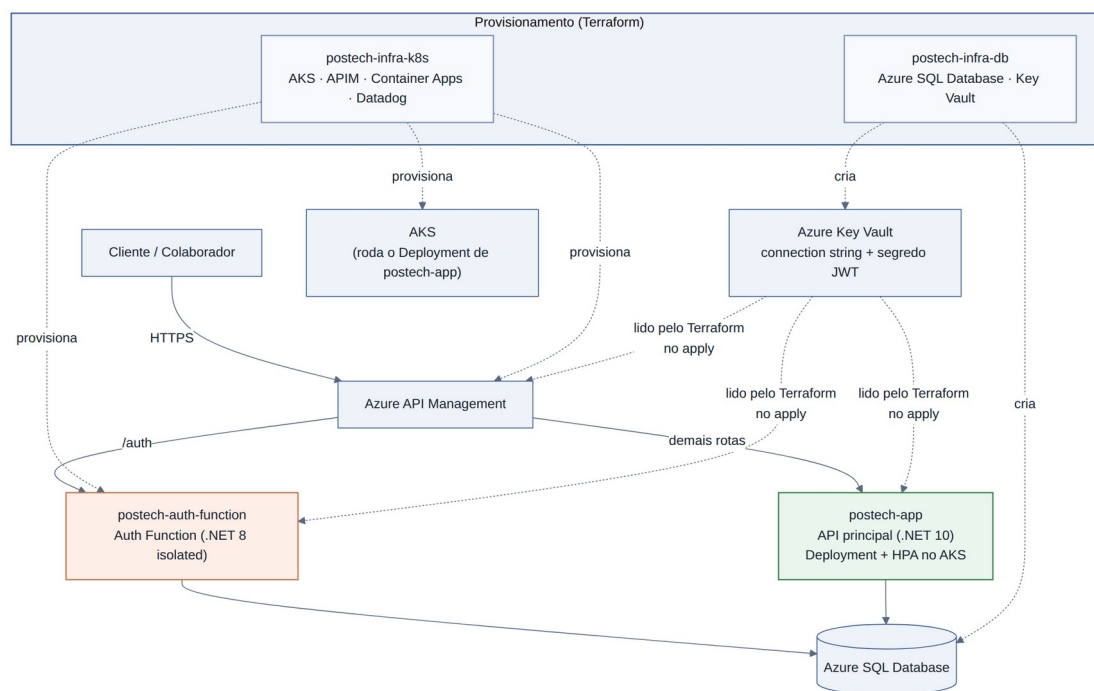


Figura 2 — Relação entre os quatro repositórios da entrega

#	Repositório	Responsabilidade
1	postech-auth-function	Function serverless (.NET 8 isolated, Azure Functions v4) que autentica o cliente por CPF e emite o JWT que a API valida
2	postech-infra-db	Terraform do Azure SQL Database e do Azure Key Vault — fonte única da connection string e da chave JWT
3	postech-app	API principal — clientes, veículos, peças, ordens de serviço, orçamentos; Deployment e HPA que rodam no AKS
4	postech-infra-k8s	Terraform do AKS, do Azure API Management, da hospedagem da Auth Function (Container Apps) e da observabilidade (Datadog)

4.3 Arquitetura de nuvem — Diagrama de Componentes

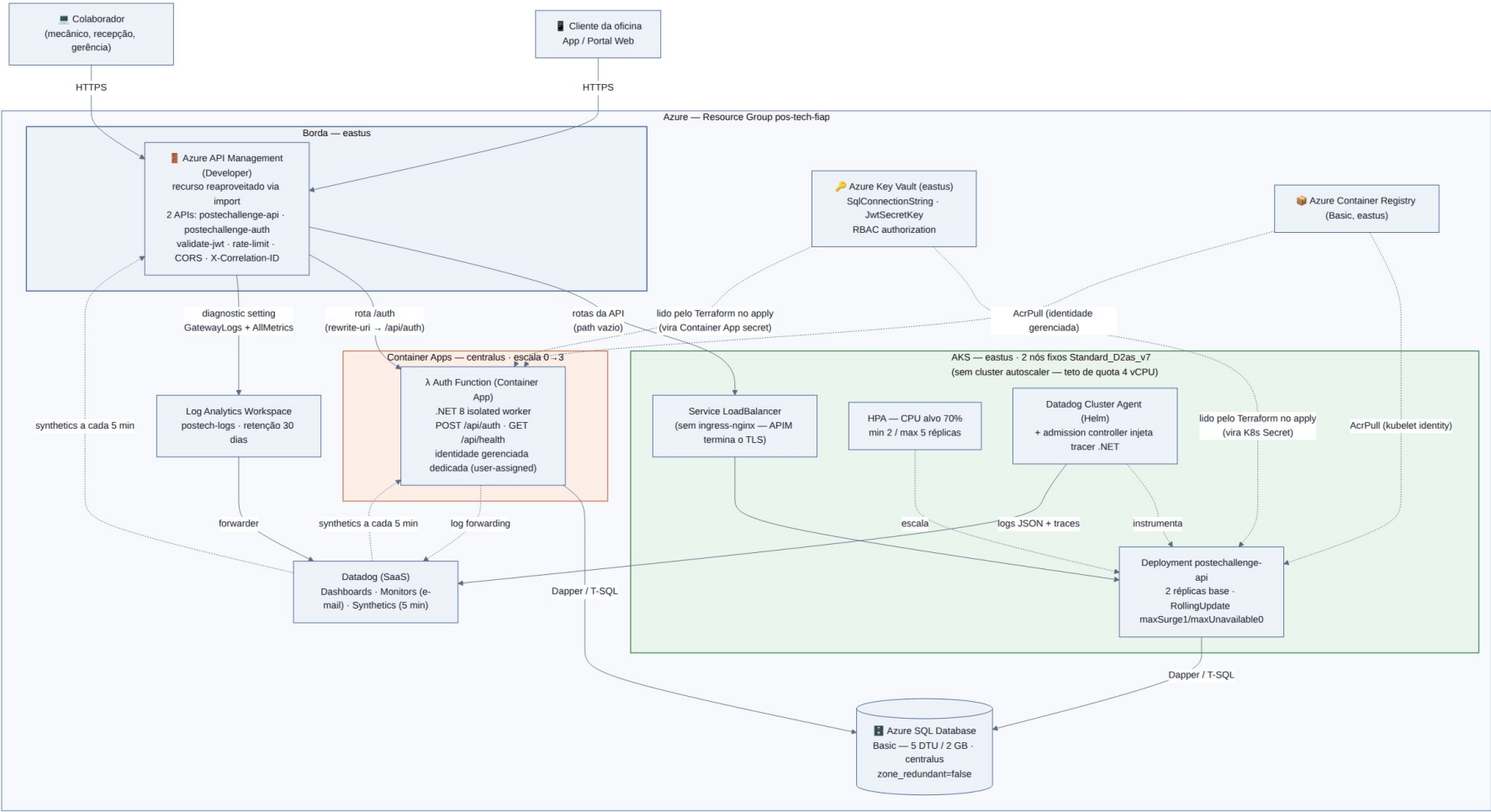


Figura 3 — Visão de componentes: Azure, APIs, banco de dados gerenciado e stack de observabilidade, como efetivamente provisionado pelos quatro repositórios.

O diagrama reflete a arquitetura efetivamente provisionada na Fase 3, na Azure (RFC-0001), dentro do resource group pos-tech-fiap de uma subscription acadêmica com quotas restritas por região — restrição que molda várias das escolhas discutidas nas RFCs adiante. Clientes finais e colaboradores acessam o sistema exclusivamente via HTTPS através do Azure API Management (SKU Developer, RFC-0004), que concentra roteamento, validação de JWT, rate limiting por IP e a propagação do cabeçalho X-Correlation-ID.

O APIM expõe duas APIs distintas: a rota /auth encaminha para o Auth Function, hospedado em Azure Container Apps em vez de um Function App clássico — a subscription tem quota zero para todos os planos de App Service testados, então Container Apps é o destino serverless viável (escala a zero, cobra por requisição; RFC-0005); as demais rotas encaminham para o Service tipo LoadBalancer do cluster AKS, que expõe diretamente os pods da aplicação, sem ingress-nginx nem cert-manager, porque o próprio APIM já termina o TLS (ADR-0003).

O cluster AKS roda com 2 nós fixos Standard_D2as_v7 e cluster autoscaler desligado: a quota regional da subscription é de 4 vCPU, exatamente o teto desses 2 nós. A escalabilidade exigida pelo desafio vem inteiramente do HPA sobre os pods (2 a 5 réplicas, CPU 70%, ADR-0002). O Datadog Cluster Agent roda como DaemonSet e, via admission controller, injeta automaticamente o tracer .NET nos pods da API, sem alterar o Dockerfile da aplicação (ADR-0004).

Tanto os pods da API quanto o Auth Function acessam a mesma Azure SQL Database (Basic, 5 DTU/2 GB) via Dapper, e ambos validam/emitem tokens com o mesmo segredo HMAC — mas nenhum dos dois lê o Azure Key Vault em tempo de execução: o Key Vault é consultado pelo Terraform no momento do apply, que materializa os segredos como Secret do Kubernetes (para os pods) e como secret do Container App (para a Function).

4.4 Diagrama de Sequência — Autenticação e Abertura de OS

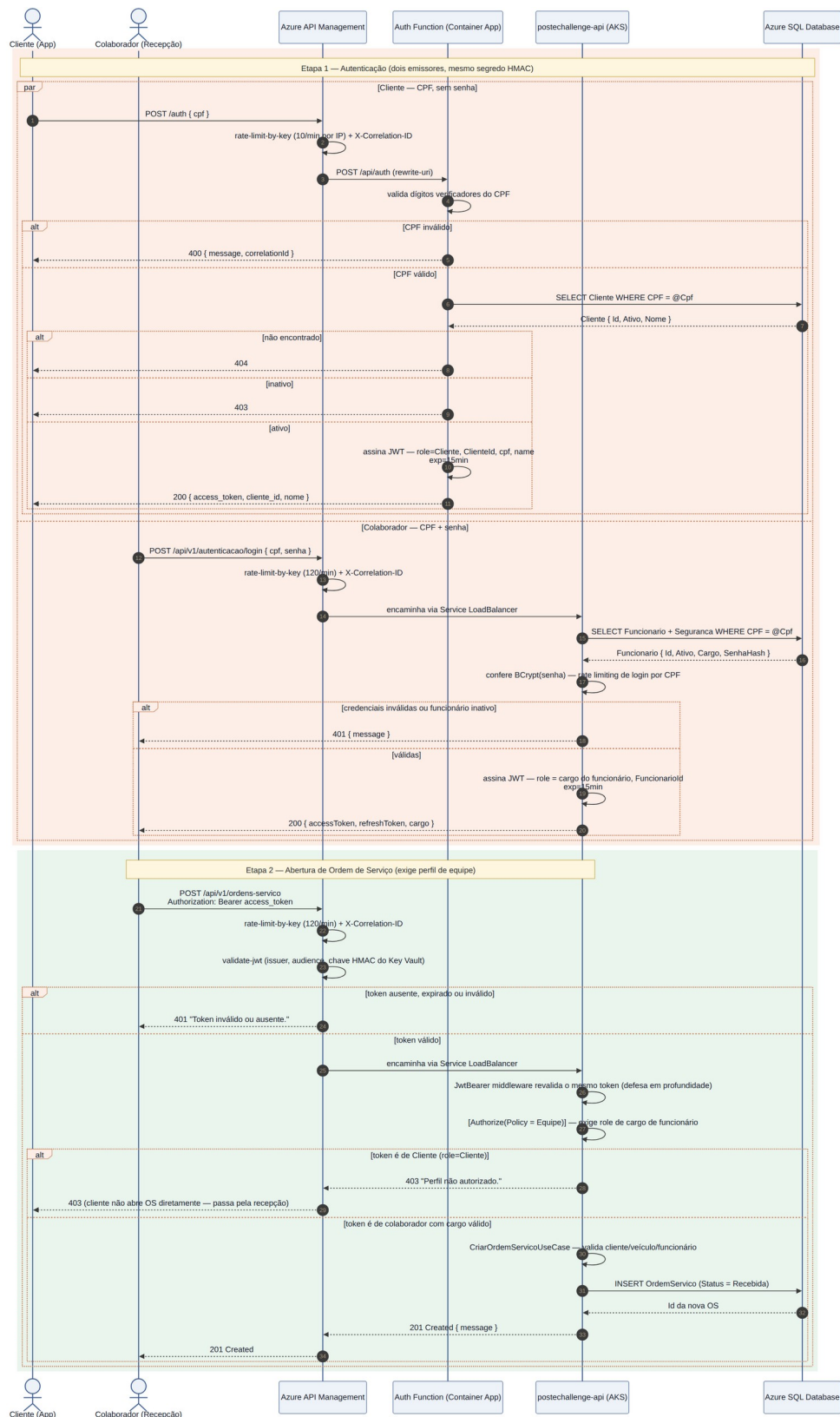


Figura 4 — Autenticação (cliente por CPF via Auth Function; colaborador por CPF + senha nesta API) seguida da abertura de OS, restrita ao perfil de equipe.

Etapa 1 — Autenticação

O sistema tem dois emissores de JWT, assinados com o mesmo segredo HMAC e validados pelo mesmo `validate-jwt` no gateway. O cliente autentica só com CPF, sem senha, em `POST /auth` — o APIM aplica um `rate limit` de 10 chamadas por minuto por IP (mais restritivo que o das rotas operacionais, porque é o único endpoint que recebe um CPF sem token algum) e encaminha ao Auth Function. A função valida o dígito verificador do CPF antes de qualquer consulta (400 se inválido), busca o cliente no Azure SQL Database e distingue três desfechos exigidos pelo desafio: 404 se o CPF não está cadastrado, 403 se o cliente existe mas está inativo, e só então a emissão do token (`role=Cliente`, `Clienteld`, `exp=15min`).

O colaborador autentica com CPF e senha em `POST /api/v1/autenticacao/login`, validado nesta própria API: a senha é conferida com BCrypt Enhanced (SHA-384) contra o hash salvo em Segurança, com `rate limiting` de tentativas por CPF. O token emitido carrega o cargo do funcionário (`role`) e seu `Funcionariold`.

Etapa 2 — Abertura da Ordem de Serviço

Abrir uma OS é uma ação da recepção, não do cliente: o endpoint `POST /api/v1/ordens-servico` exige a policy Equipe — qualquer cargo de funcionário (a lista é derivada do enum de cargos, não escrita à mão, para que nenhum cargo novo fique de fora por engano). Um token de cliente é rejeitado com 403 nesta rota, ainda que válido — o cliente acompanha e aprova sua OS, mas não a abre diretamente.

A validação se repete no lado da aplicação — o middleware `JwtBearer` revalida o mesmo token — uma defesa em profundidade deliberada: mesmo que alguém alcance o Service diretamente, sem passar pelo APIM, a API ainda exige um token válido e com o papel correto. Uma vez autorizado, o controller delega ao `CriarOrdemServicoUseCase`, que confere a consistência do pedido (cliente, veículo e funcionário) e insere a OS com status inicial Recebida.

Este fluxo não grava nada na tabela `EmailOutbox`: no código atual, o Outbox só é usado pelo caso de uso de envio de orçamento, não pela abertura de OS — e mesmo ali existe apenas o lado produtor, sem um processo que efetivamente envie o e-mail (ver ADR-0001).

4.5 Fluxo de deploy (CI/CD)

O pipeline da API principal (`.github/workflows/ci-cd.yml`) autentica na Azure via OIDC — sem `client secret` armazenado no repositório — e fecha com um smoke test que valida o ambiente publicado de ponta a ponta, contra o gateway real.

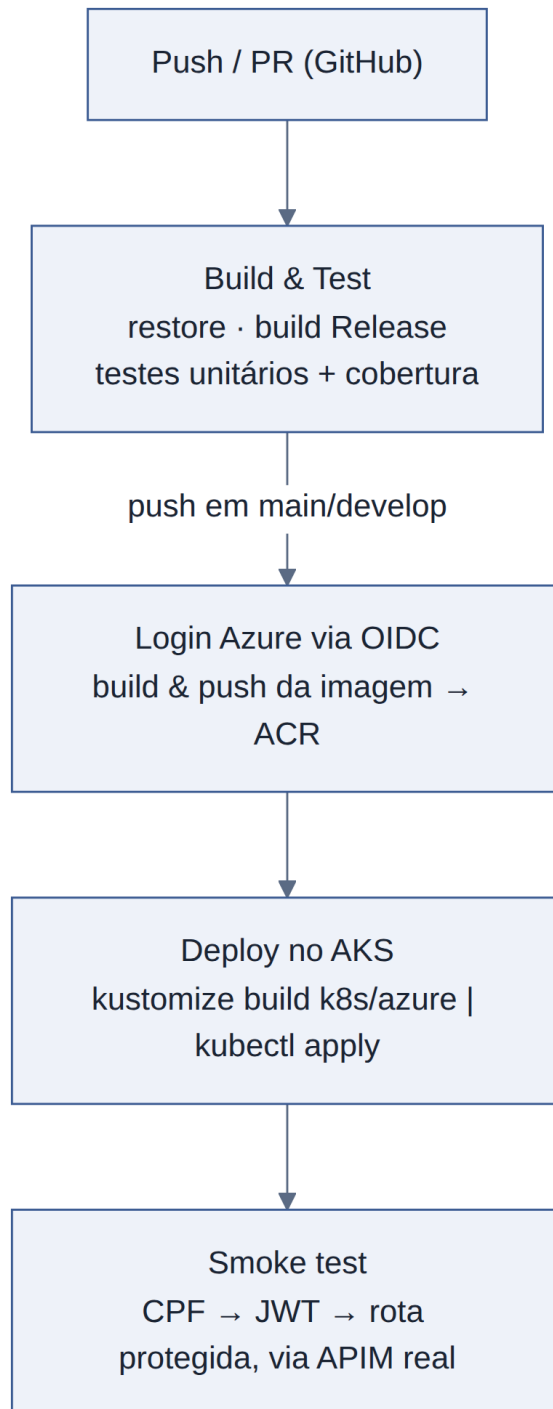


Figura 5 — Etapas do pipeline de CI/CD da API principal

Build & Test roda em todo push e PR: restaura, builda em Release e executa os testes unitários, publicando relatório e cobertura como artefato. Deploy roda apenas em push a main/develop: builda e publica a imagem no ACR, aplica ConfigMap + Deployment + HPA no AKS via Kustomize e aguarda o rollout. O smoke test final valida, contra o APIM real: /health do cluster, Swagger habilitado, 401 sem token numa rota protegida, autenticação de cliente por CPF na Auth Function (com tolerância ao cold start do Container App), 400 para CPF inválido, 403 quando um token de cliente tenta uma rota de equipe, acesso liberado às próprias ordens do cliente, e login de funcionário liberando a rota de equipe.

Os outros três repositórios têm pipelines próprios e independentes: postech-auth-function publica sua imagem no ACR; postech-infra-db e postech-infra-k8s rodam fmt/init/validate/plan em Pull Request (com o plano comentado no PR) e apply na main, também via OIDC. A pipeline de postech-infra-k8s confirma, antes do apply, que a imagem da Function já existe no ACR — se não existir, ela para com uma mensagem clara em vez de falhar no meio do provisionamento.

5. RFC's — Request for Comments

Registradas aqui as decisões que envolveram alternativas de mercado — escolhas de fornecedor ou produto.

RFC-0001	Escolha do Provedor de Nuvem	Status: Aceita e implementada
----------	------------------------------	-------------------------------

Contexto

O desafio permite livre escolha do provedor de nuvem. O time tinha crédito educacional disponível na Azure e experiência prévia com o portal, o que pesou na decisão em relação a AWS e GCP.

Decisão

Publicar toda a solução na Azure, dentro do resource group pos-tech-fiap de uma subscription acadêmica.

Consequências

- A subscription tem quotas reduzidas por região (vCPU, planos de App Service, disponibilidade regional de Azure SQL) — restrição que moldou diretamente as RFC-0005, ADR-0002 e ADR-0003;
- Recursos como o API Management foram reaproveitados via import do Terraform em vez de recriados, para não pagar de novo o tempo de provisionamento (~40 minutos no caso do APIM Developer).

RFC-0002	Escolha do Banco de Dados Gerenciado	Status: Aceita e implementada
----------	--------------------------------------	-------------------------------

Contexto

O sistema precisa de um banco relacional gerenciado, compatível com o schema e as queries Dapper/T-SQL já existentes da Fase 2, dentro das quotas regionais disponíveis na subscription.

Decisão

Azure SQL Database, tier Basic (5 DTU / 2 GB). O comparativo formal com alternativas (PostgreSQL e MySQL Flexible Server) e a justificativa detalhada estão na Seção 7.

Consequências

- O tier Basic tem um teto de conexões simultâneas e de DTU que não sustentaria uma carga de produção real, mas é suficiente para a demonstração do desafio;
- Azure SQL não estava disponível nas regiões eastus/eastus2 desta subscription — o banco roda em centralus, o que colocou favoravelmente o Auth Function (também em centralus, por RFC-0005).

Contexto

O desafio exige uma função serverless dedicada para autenticar o cliente por CPF, sem senha. A Fase 2 já tinha login de funcionário por CPF + senha, que precisava conviver com o novo mecanismo sem duplicar a lógica de emissão de token.

Decisão

Dois emissores de JWT — a Auth Function para o cliente (CPF apenas) e a própria API para o funcionário (CPF + senha) — assinados com o mesmo segredo HMAC (Azure Key Vault) e aceitos pelo mesmo validate-jwt do gateway. A claim role distingue os dois; a policy Equipe exige um cargo de funcionário; e a checagem de posse (ClienteAcessandoOutro / ClienteAcessandoOrdemDeOutroAsync) garante que um cliente autenticado só acesse os próprios dados.

Consequências

- Defesa em profundidade: o APIM valida o JWT antes de encaminhar a chamada, e a API revalida o mesmo token de forma independente;
- A abertura de OS exige o perfil de equipe — o cliente acompanha e aprova sua ordem, mas não a abre diretamente, o que reflete o processo real de recepção da oficina;
- O corpo da resposta dos dois fluxos usa convenções de nomes diferentes (access_token/cliente_id no Function vs. accessToken/refreshToken na API) — um cliente HTTP que consome os dois precisa tratar essa diferença.

RFC-0004	Escolha do API Gateway	Status: Aceita e implementada
----------	------------------------	-------------------------------

Contexto

A solução precisa de um ponto único de entrada que valide o JWT, aplique rate limiting e propague um identificador de correlação, na frente tanto da API quanto da Auth Function.

Decisão

Azure API Management, SKU Developer — um recurso já existente na subscription, importado para o state do Terraform (com `prevent_destroy`) em vez de recriado.

Consequências

- Duas APIs publicadas no mesmo gateway (postechallenge-api e postechallenge-auth), cada uma com sua política de rate limiting e roteamento próprios;
- O SKU Developer não tem SLA de produção nem alta disponibilidade — adequado para a demonstração do desafio, não para uma carga real.

Contexto

O destino padrão para uma Azure Function é um Function App, hospedado sobre um plano de App Service. A subscription usada na entrega retornou quota zero para todos os planos testados — Y1, F1, B1, S1, P0v3 e P1v3 — tornando essa opção padrão inviável.

Decisão

Hospedar o Auth Function em Azure Container Apps, rodando a imagem base oficial do Azure Functions (.NET 8 isolated worker) — o código, os triggers e os bindings são exatamente os de uma Function normal; muda apenas o plano de hospedagem. O ambiente de Container Apps ficou em centralus, a única região com quota disponível para esse recurso nesta subscription.

Consequências

- A identidade do Container App é gerenciada pelo usuário (user-assigned), não a identidade de sistema padrão — necessária porque o app precisa da permissão AcrPull já na primeira revisão para puxar a imagem;
- Escalar a zero significa que a primeira requisição após um período ocioso paga o custo de cold start do host de Functions.

6. ADRs — Architecture Decision Records

Registradas aqui as decisões arquiteturais consideradas permanentes para o time, independentes de qual fornecedor específico as implementa.

ADR-0001	Padrão de comunicação — Transactional Outbox	Status: Aceito — parcialmente implementado
----------	--	--

Contexto

O envio de e-mail ao cliente (aprovação de orçamento) não deve bloquear nem falhar a transação de negócio que o originou.

Decisão

Gravar a mensagem em uma tabela EmailOutbox na mesma transação do caso de uso, desacoplando o envio do fluxo transacional.

Consequências

- Nesta entrega existe apenas o lado produtor (EnviarOrçamentoUseCase grava na tabela) — nenhum dos quatro repositórios contém um processo consumidor que efetivamente envie o e-mail. É um passo natural de uma próxima iteração;
- A abertura de OS não usa o Outbox em nenhum momento — apenas o envio de orçamento.

ADR-0002	Escalabilidade — HPA sobre nós fixos	Status: Aceito
----------	--------------------------------------	----------------

Contexto

O desafio exige escalabilidade dinâmica baseada em consumo de recursos. A quota regional da subscription é de 4 vCPU.

Decisão

Cluster AKS com 2 nós fixos Standard_D2as_v7 (4 vCPU no total, o teto da quota), sem cluster autoscaler. A escalabilidade acontece inteiramente no nível de pod, via HPA (2 a 5 réplicas, CPU alvo 70%, janelas de estabilização de 30s no scale-up e 120s no scale-down).

Consequências

- 5 réplicas a 150m de request cabem com folga nos 4 vCPU disponíveis, ao lado do kube-system e do agente do Datadog;
- Um pico de carga que exigisse mais do que 5 pods, ou mais do que 2 nós, esbarraria na quota — não há caminho de escala horizontal além desse teto nesta subscription.

ADR-0003	Service LoadBalancer em vez de Ingress	Status: Aceito
----------	--	----------------

Contexto

A aplicação precisa de um ponto de entrada no cluster para o APIM encaminhar as requisições.

Decisão

Expor a API por um Service do tipo LoadBalancer, sem ingress-nginx nem cert-manager — o próprio APIM já termina o TLS antes do tráfego chegar ao cluster, então um Ingress adicionaria uma camada sem função. O Service é criado pelo Terraform de postech-infra-k8s, e não pela aplicação, porque o APIM precisa do IP dele como backend antes dos pods subirem.

Consequências

- O tráfego entre o APIM e o cluster é HTTP simples — HttpsRedirection fica desligado no ambiente Azure;
- Trocar de estratégia (por exemplo, para Ingress com múltiplos hosts) exigiria coordenar uma mudança em dois repositórios ao mesmo tempo.

ADR-0004	Ferramenta de observabilidade — Datadog	Status: Aceito
----------	---	----------------

Contexto

O desafio exige monitoramento com painéis de volume de OS, tempo por status, erros de integração, latência e consumo de recursos.

Decisão

Datadog: Cluster Agent (Helm) com admission controller injetando o tracer .NET automaticamente nos pods, log forwarding nativo do Container App para o Auth Function, e um forwarder lendo o Log Analytics Workspace para onde o APIM exporta GatewayLogs. Dois testes de synthetics complementam a cadeia, batendo no /health do gateway e da Function a cada 5 minutos.

Consequências

- Nenhuma alteração no Dockerfile da aplicação foi necessária — a instrumentação acontece via admission controller no cluster;
- O Auth Function, por rodar em Container Apps (sem DaemonSet), depende do log forwarding nativo em vez do Cluster Agent — um caminho de observabilidade diferente do resto do sistema.

ADR-0005	Segregação em quatro repositórios	Status: Aceito
----------	-----------------------------------	----------------

Contexto

O desafio exige quatro repositórios — Function, infraestrutura de banco, infraestrutura de Kubernetes e aplicação —, cada um com CI/CD próprio, branch main protegida e Pull Request obrigatório.

Decisão

Adotar a segregação exigida: postech-auth-function, postech-infra-db, postech-app e postech-infra-k8s, cada um autenticando na Azure por OIDC (sem segredo de cliente armazenado). O contrato entre eles é o Azure Key Vault (criado por postech-infra-db, fonte única da connection string e do segredo JWT) e o Azure Container Registry (criado por postech-infra-k8s, onde postech-app e postech-auth-function publicam suas imagens).

Consequências

- Isolamento de responsabilidade e de blast radius, com uma dependência de ordem entre pipelines: banco antes de Key Vault, Key Vault antes de infra-k8s, imagem publicada antes do apply de infra-k8s;
- prevent_destroy está ativo tanto no APIM quanto no banco de dados, para que um terraform destroy disparado no repositório errado não os leve junto.

7. Banco de Dados — Justificativa e Modelo Relacional

7.1 Comparativo formal entre motores

Critério	Azure SQL Database	PostgreSQL Flexible Server	MySQL Flexible Server
Compatibilidade com o schema/queries existentes	Total — T-SQL nativo, sem tradução	Exigiria portar toda a camada Dapper/T-SQL	Idem
Tiers de baixo custo	Basic (5 DTU/2GB), sob demanda acadêmica	Burstable B1ms, similar	Burstable B1s, similar
Disponibilidade regional nesta subscription	Bloqueado em eastus/eastus2; disponível em centralus	Disponível nas regiões testadas	Disponível nas regiões testadas
Curva de adoção do time	Já era o motor da Fase 2 (SQL Server)	Nova, exigiria retrabalho de queries	Nova, exigiria retrabalho de queries

7.2 Decisão

Manter Azure SQL Database, no tier Basic (5 DTU / 2 GB). A compatibilidade total com o schema e as queries Dapper/T-SQL herdados da Fase 2 elimina o custo e o risco de portar a camada de persistência sob o prazo do desafio. O bloqueio de provisionamento em eastus/eastus2 nesta subscription acadêmica moveu o banco para centralus — decisão que, como efeito colateral favorável, colocou o banco com o Auth Function (RFC-0005), também hospedado em centralus pela mesma razão de quota.

7.3 Ajustes no modelo relacional

O schema efetivamente aplicado nesta entrega (postech-infra-db/sql/schema-azure.sql) é, na quase totalidade, o mesmo da Fase 2. A única mudança real desde a última revisão deste documento foi a coluna Ativo em Funcionario, adicionada para permitir desativação em vez de exclusão — sem ela, a rota de exclusão de funcionário nunca funcionava, porque a tabela é referenciada por Seguranca, OrdemServico, Itens e Status, nenhuma com cascade, e um DELETE de verdade sempre esbarrava na FK do login.

Os demais ajustes abaixo continuam como recomendação formal para um próximo incremento, não como descrição do que já existe implementado. A lacuna mais relevante é estrutural: o próprio desafio da Fase 3 parte da premissa de que a oficina expandiu para múltiplas unidades físicas, mas o schema não tem nenhuma tabela ou coluna que represente uma unidade — a modelagem continua implicitamente de oficina única.

- Nova entidade Unidade (filial): nenhuma tabela do schema atual representa uma unidade física. É a lacuna mais direta em relação à premissa de negócio da Fase 3;
- FK UnidadeId em Funcionario: cada colaborador passaria a estar lotado em uma unidade específica;
- FK UnidadeId em OrdemServico: cada OS passaria a registrar em qual unidade foi executada, sustentando relatórios operacionais por filial;

- Separação de Peças (catálogo) e EstoquePeca (saldo por unidade): hoje a tabela Pecas mistura atributos de catálogo com um único contador global de QuantidadeEstoque — que deixa de fazer sentido assim que existe mais de uma oficina física com estoque próprio;
- Constraint UNIQUE em Cliente.CPF, Cliente.CNPJ e Funcionario.CPF: continuam ausentes no schema publicado, o que permite cadastros duplicados do mesmo documento. Isso ganha um agravante concreto na Fase 3: a autenticação de clientes por CPF (RFC-0003) depende de o CPF identificar um único cliente de forma inequívoca — o Auth Function faz um SELECT ... WHERE CPF = @Cpf esperando no máximo uma linha, e o schema atual não garante isso;
- CHECK constraint em Cliente garantindo exatamente um entre CPF e CNPJ preenchido — ambos continuam opcionais sem essa regra;
- CHECK constraint em Itens garantindo exclusividade entre IdFuncionario e IdPeca conforme Tipoltem — inconsistências hoje dependem inteiramente da disciplina da camada de aplicação;
- Índices não clusterizados IX_OrdemServico_Status e IX_OrdemServico_CriadoEm: sustentariam diretamente dois dos painéis do dashboard já em produção no Datadog — volume diário de OS e tempo médio por status —, hoje alimentados sem esse suporte de índice à medida que o volume cresce.

7.4 Diagrama Entidade-Relacionamento

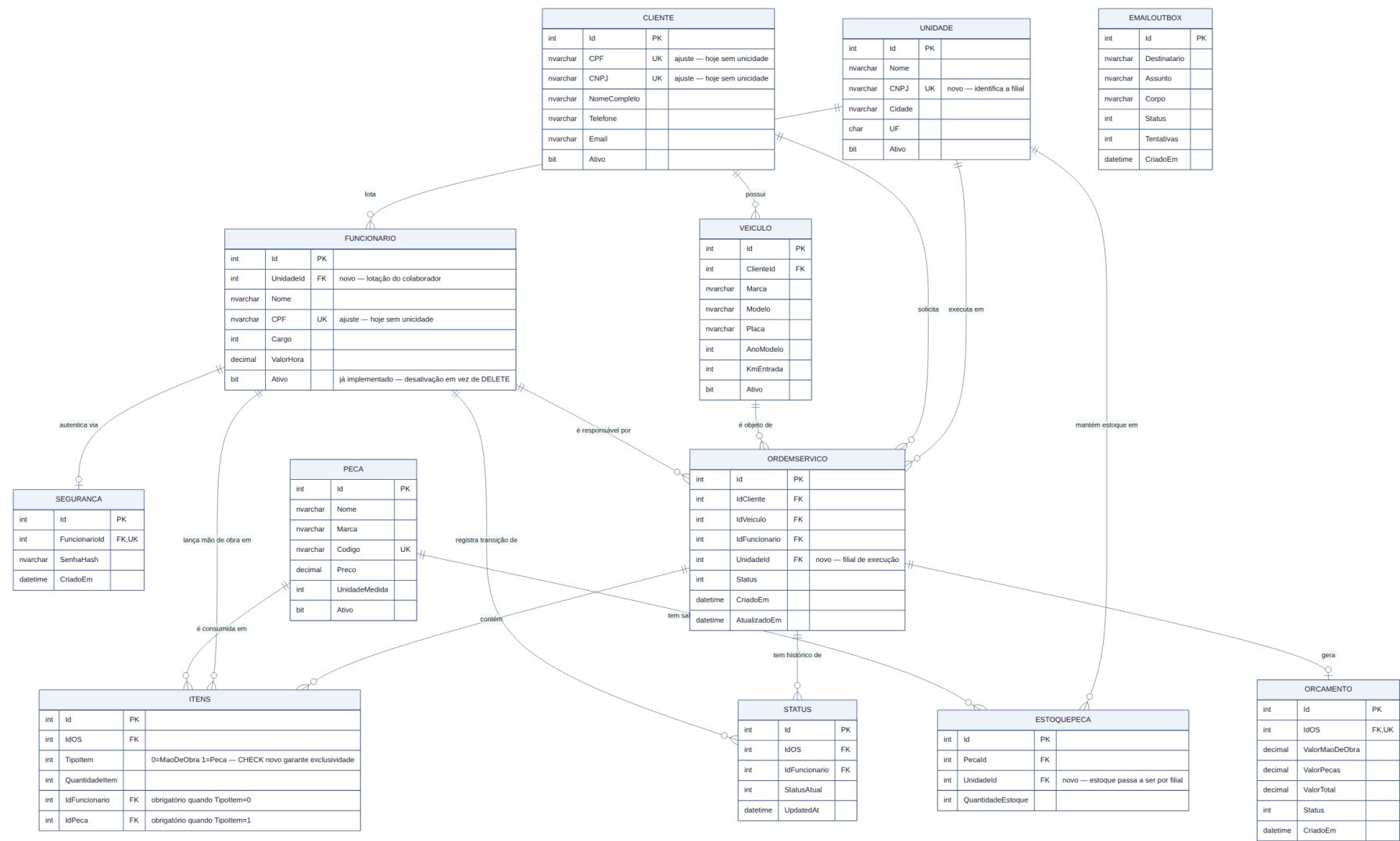


Figura 6 — Modelo relacional: em preto, o schema como publicado (com a coluna Ativo de Funcionario já implementada); as anotações marcam os ajustes que permanecem como recomendação.

7.5 Explicação dos relacionamentos

Relacionamento	Cardinalidade	Situação
Unidade → Funcionário / OrdemServico / EstoquePeca	1:N	Proposto — depende da entidade Unidade
Funcionário → Segurança	1:1	Já implementado — credenciais de login
Cliente → Veículo / OrdemServico	1:N	Já implementado
Veículo → OrdemServico	1:N	Já implementado
Funcionário → OrdemServico / Itens / Status	1:N	Já implementado
OrdemServico → Itens / Status / Orçamento	1:N / 1:N / 1:1	Já implementado
Peca → Itens / EstoquePeca	1:N	EstoquePeca proposto (hoje é coluna em Pecas)

7.6 Migração

Os ajustes propostos seguem o mesmo padrão de script idempotente já usado no schema atual (verificações IF NOT EXISTS antes de cada CREATE/ALTER), para que a migração possa ser reaplicada com segurança sobre um ambiente já provisionado, sem exigir uma janela de manutenção coordenada entre os quatro repositórios.

8. APIs da Fase 3

Além das APIs de gestão herdadas das fases anteriores (clientes, veículos, funcionários, peças, itens de OS e orçamentos), esta fase introduziu ou alterou os seguintes contratos:

Requisito	Endpoint	Observações
Autenticação de cliente por CPF (função serverless)	POST /auth (via APIM) → postech-auth-function	Sem senha; distingue 400 (CPF inválido), 404 (não cadastrado) e 403 (inativo) antes de emitir o token
Login de funcionário (herdado, agora atrás do gateway)	POST /api/v1/autenticacao/login	CPF + senha, BCrypt Enhanced (SHA-384), rate limiting por CPF
Abertura de Ordem de Serviço	POST /api/v1/ordens-servico	Passou a exigir a policy Equipe — um token de cliente recebe 403
Consulta de status da OS pelo cliente	GET /api/v1/ordens-servico/{id}/status	Deixou de ser público (Fase 2); agora exige token e checagem de posse (ClienteAcessandoOutro)
Resposta a orçamento pelo cliente	POST /api/v1/orcamentos/os/{idOS}/responder	Deixou de ser público; exige token válido, para que só o dono do orçamento possa responder por ele
Listagem ordenada por status	GET	Inalterada — prioridade Em

Requisito	Endpoint	Observações
	/api/v1/ordens-servico/ordenado-por-status	Execução > Aguardando Aprovação > Diagnóstico > Recebida, mais antigas primeiro

O fechamento de rotas antes públicas ou aceitando qualquer token autenticado — status da OS, resposta de orçamento, consulta de valor de OS, entre outras — foi o principal endurecimento de segurança desta fase, registrado em detalhe na RFC-0003. A regra aplicada em todo o sistema é uniforme: rota de operação exige a policy Equipe; rota que o próprio cliente legitimamente usa ganha checagem de posse.

Ciclo de vida da OS, inalterado desde a Fase 2: Recebida → Em Diagnóstico → Aguardando Aprovação → Em Execução → Finalizada → Entregue, com transições estritamente sequenciais validadas no domínio (OrdemServicoDomainService).

9. Como Executar

As instruções completas e detalhadas estão no README.md de cada repositório. Resumo dos caminhos disponíveis:

9.1 Execução local (Docker Compose)

- `docker compose up -d --build` no repositório `postech-app` — sobe API + SQL Server com schema e seed; Swagger em `http://localhost:8080/swagger`. Este caminho não depende de nenhum dos outros três repositórios.

9.2 Deploy em Kubernetes local (kind)

- Script automatizado `local-k8s-up.bat` (Windows/Docker Desktop), ou passo a passo manual com `kind + kubectl apply -k k8s/overlays/local` — reaproveita o modelo da Fase 2, com SQL Server como `StatefulSet` dentro do próprio cluster.

9.3 Provisionamento local com Terraform

- `cd infra/terraform && terraform init && terraform apply`, no repositório `postech-app` — cria o cluster `kind`, os `Secrets`, aplica os manifestos e inicializa o banco. Variáveis sensíveis via `terraform.tfvars` (não versionado) ou `TF_VAR_*`.

9.4 Ambiente publicado (Azure)

- O ambiente de produção já está de pé, provisionado pelos repositórios `postech-infra-db` e `postech-infra-k8s` e mantido pelos pipelines de CI/CD dos quatro repositórios — não há passo manual de deploy. Uma alteração em qualquer um dos quatro é publicada automaticamente ao chegar em `main`, incluindo o smoke test contra o APIM real descrito na Seção 4.5.

10. Vídeo Demonstrativo

Link: <https://www.youtube.com/watch?v=Awv2vRF1krI>